

川口演習 2025 秋

GitHub の R コード実行に当たっての Tips

©松前景一郎

2025/9/26

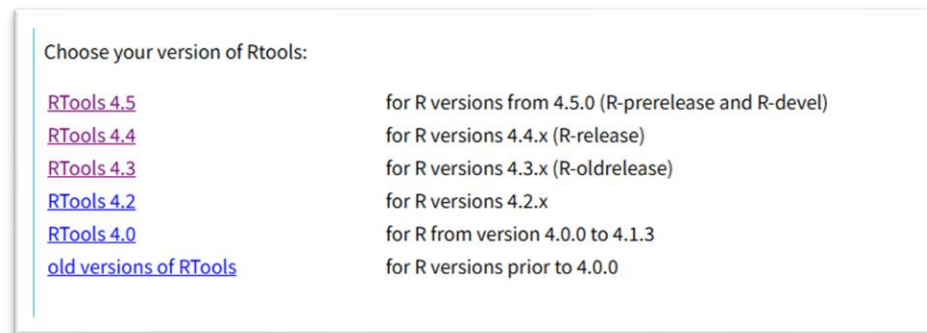
- 対象コードは以下の Github 上のコード（教科書で指定の Code）
[spbook_jp/README.md at main · dmuraka/spbook_jp · GitHub](https://github.com/dmuraka/spbook_jp/blob/main/README.md)
- 前提
 - 以下の環境に基づいて試行した結果に基づく Tips です
 - パソコン
 - OS : Windows11Pro
 - プロセッサ 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz、2803 Mhz、4 個のコア、8 個のロジカル プロセッサ
 - インストール済みの物理メモリ (RAM) 32.0 GB
 - R : R version 4.5.1 (2025-06-13 ucrt)
 - RTools : Version 4.5
 - RStudio: 2025.05.0 Build 496
- General tips（全セクションに関係する Tips）
 - Package インストールにそれなりの時間がかかる場合があります：Package インストールが想像以上に時間がかかる場合があるので（PC のスペックにもよりますが、32GB メモリの PC でも数十分など必要とする場合があります）、授業前に必ず対象 R コードを 1 度は回すことを推奨します。
 - 上記に関連して、以下のポップアップについては、特別の理由がなければ「いいえ」を選択する。これにより R コードがこけない場合があります。CRAN などからダウンロードするものはすでにバイナリ版です。バイナリ版とはパッケージのソースコード（人間が読んで理解できるプログラム）を機械語（2 進数の集まり）に変換したものです。そのため、ここまでのコンソール表示上ではバイナリよりソースの方が Version が新しいように見られますが、ここで「はい（＝ソースからコンパイル）」を選択すると長時間（数分から数十分）待たされた上にコードがこけるなどがありました（Windows では RTools をインストールしていないとうまくいきません。RTools をインストールしていてもうまく動作しない場合もあります）。

す)。



- **Package のインストールと以降の code を別々に実行することによりエラーが避けられる場合があります：**各 Code の冒頭の「install.packages(“*Package 名*”)」だけで以降の code とは別に先に実行することによりエラーが避けられる場合があります。すなわち、Github ないし本文書後段にある code を実行する際に、「install.packages(“*Package 名*”)」と以降の code を含めて一度に実行してエラーになったら、別々に実行することによりエラーが避けられる場合があります。その理由は、R 言語はインタプリタ方式でありバッチ方式ではないからです。インタプリタ方式ではコードを 1 行ずつ実行します。これに対して、バッチ方式では全コードを一括して実行します。R 言語は前者なので 1 つのコードの実行に時間を要する場合、複数のコードを一括実行するとエラーになります。
- **Package インストール以外でも時間がかかるコードがあります：**Package インストール以外でも、以下のコードは恐ろしく（正確に測ってないですが 30 分等）時間がかかりましたので、余裕をもって実行ください。下記の修正版コード対象ですが、修正版でも同様に時間がかかります。ちなみに、この分野の研究では数日から 1 週間程度の時間を要することがあります。
 - 上記の Github の Secion18 のコード→ [spbook_jp/n_Section 18 \(ST-CAR model\) at main · dmuraka/spbook_jp · GitHub](https://github.com/dmuraka/spbook_jp/blob/main/n_Section%2018%20(ST-CAR%20model).R)
- **RTools：**RTools は Windows 環境で R のパッケージをソースからインストールとコンパイル（機械語、バイナリ版に変換）するためのツールです（ご自分の R の Version に合った RTools を入れる必要があります）。ちなみに、Mac では Rtools は必要ありません。MacOS には R のパッケージをソースからインストールとコンパイルするための環境が最初から提供されています。

- [RTools: Toolchains for building R and R packages from source on Windows](#)



- Code によっては `install.packages` だけ行って止まるパターンが見られました（色々試行錯誤していたので全部は記憶してないですが少なくとも「**o_Section 19 (ST-geostatistical model))**」など）。その場合は `install.packages` 以下を再実行すると成功しました

- 修正版 R Code 説明 (R Code そのものはスクロールした先にあります)
 - 全体：以下が修正対象リストとその適用です。具体的な修正済コードは当資料をさらに下へスクロールした箇所に記載しています。
 - **d_Section 7 (GISA)および e_Section 8 (LISA)**
 1. rgdal パッケージが 2023 年 10 月 16 日をもって Archived (という CRAN 上の扱い) になった模様です
 2. つきまして、rgdal の代替として sf パッケージを採用しています
 > install.packages(c("sf", "spdep", "RColorBrewer", "NipponMap"))
 - **g_Section 10 (CAR model)**
 1. 筆者が試した時点の後に、R のパッケージ側で変更が入ったことにより不整合が起きていると思われます。具体的には以下の修正を入れています。
 > shp -> .gpkg に変更。layer を明示
 > boston.tr <- st_read(system.file("shapes/boston_tracts.gpkg",
 package="spData"), layer = "boston_tracts")
 - **m_Section 16 (INLA)**
 1. 失敗しにくい導入 (timeout 延長・安定版優先・必要パッケージ自動導入) を追加。
 2. 本編ロジックはほぼ不変で、最小限の型・仕様適合 (y をベクトル化、alpha を整数 2) だけ修正。

修正版 R Code

① d_Section 7 (GISA)

```
install.packages(c("sf", "spdep", "RColorBrewer", "NipponMap"))
library(sf)
library(spdep)
library(RColorBrewer)
library(NipponMap)

##### Prepare prefecture polygon data in Japan (same as the previous section)
shp      <- system.file("shapes/jpn.shp", package = "NipponMap")[1]
pref0    <- read_sf(shp)
pref0b   <- pref0[pref0$name != "Okinawa",]
st_crs(pref0b) <- 4326
pref     <- st_transform(pref0b, crs=6677)

##### Adjacency matrix
coords   <- st_coordinates(st_centroid(pref))
knn      <- knearneigh(coords, 4)
nb2      <- knn2nb(knn)
w        <- nb2listw(nb2)

##### Population data
pop      <- pref$population

##### Moran's I statistic
moran    <- moran.test(pop, listw=w)
moran

##### Geary's C statistic
geary    <- geary.test(pop, listw=w)
geary

##### (Global) Getis G statistic
w_b      <- nb2listw( nb2, style="B" )
G        <- globalG.test( pop, listw = w_b )
G
```

```
##### (Global) Getis G* statistic  
w_b2    <- nb2listw(include.self(nb2),style="B")  
Gstar    <- globalG.test(pop, listw=w_b2)  
Gstar
```

② e_Section 8 (LISA)

```
# install.packages(c("sf", "spdep", "RColorBrewer", "NipponMap"))
library(sf)
library(spdep)
library(RColorBrewer)
library(NipponMap)

##### Prepare prefecture polygon data in Japan (same as the previous section)
shp      <- system.file("shapes/jpn.shp", package = "NipponMap")[1]
pref0    <- read_sf(shp)
pref0b   <- pref0[pref0$name != "Okinawa",]
st_crs(pref0b) <- 4326
pref     <- st_transform(pref0b, crs=6677)
coords   <- st_coordinates(st_centroid(pref))
knn       <- knearneigh(coords, 4)
nb2       <- knn2nb(knn)
w         <- nb2listw(nb2)
pop       <- pref$population

##### Local Moran's I statistic #####
lmoran    <- localmoran(pop, listw=w)
lmoran[1:5,]

##### Plot the local Moran's I
pref$lmoran <- lmoran[, "Ii"]
breaks     <- c(-5, -2, -1, -0.5, 0, 0.5, 1, 2, 5)
nc         <- length(breaks)-1
pal        <- rev(brewer.pal(n = nc, name = "RdYlBu"))
plot(pref[, "lmoran"], pal = pal, breaks=breaks)

##### Plot the statistical significance
pref$lmoran_p <- lmoran[, "Pr(z != E(Ii))"] # This line might not work for old version of the
spdep package
breaks      <- c(0, 0.01, 0.05, 0.10, 1)
nc          <- length(breaks)-1
```

```
pal          <- rev( brewer.pal(n = nc, name = "YlOrRd" ))
plot(pref[, "lmoran_p"], pal = pal, breaks=breaks)
```

```
##### Moran scatter plot
```

```
moran.plot(pop, listw=w, labels=pref$name, pch=20,xlim=c(-
1000000,13000000),ylim=c(1000000,8000000))
```

```
##### Local Getis/Ord G* statisti c#####
```

```
w_b2        <- nb2listw(include.self(nb2))
```

```
lG           <- localG(pop, listw=w_b2)
```

```
lG[1:5]
```

```
##### Plot the statistical significance
```

```
pref$lG<- lG
```

```
breaks <- c(-5, -2.58, -1.96, -1.65, 0, 1.65, 1.96, 2.58, 5)
```

```
nc          <- length(breaks)-1
```

```
pal         <- rev(brewer.pal(n = nc, name = "RdYlBu"))
```

```
plot(pref[, "lG"], pal=pal, breaks=breaks)
```


③ g_Section 10 (CAR model)

```
# install.packages( c("spData", "CARBayes", "sf", "RColorBrewer") )
library(spdep)
library(CARBayes)
library(sf)
library(RColorBrewer)

boston.tr <- st_read(system.file("shapes/boston_tracts.gpkg", package="spData"),
layer="boston_tracts")
w_nb      <- poly2nb( boston.tr )
W         <- nb2mat(w_nb,style="B")

formula   <- log(CMEDV) ~ CRIM + I(RM^2) + log(LSTAT)
resLm     <- S.glm(formula, data=boston.tr, family="gaussian", burnin=5000,
n.sample=20000)
resLm

resLex    <- S.CARleroux(formula, data=boston.tr, family="gaussian", W=W,
burnin=5000, n.sample=20000)
resLex

resCAR    <- S.CARleroux(formula, data=boston.tr, rho=1, family="gaussian", W=W,
burnin=5000, n.sample=20000)
resCAR

boston.tr$predLm <- exp( resLm$fitted.values )
boston.tr$predLex<- exp( resLex$fitted.values)
boston.tr$predCAR<- exp( resCAR$fitted.values)

nc        <- 10
breaks    <- seq(0, 55, len=nc+1)
pal       <- rev(brewer.pal(nc, "RdYlBu"))
plot(boston.tr[, "CMEDV"] ,pal=pal,breaks=breaks, axes=TRUE,lwd=0.1)
plot(boston.tr[, "predLm"] ,pal=pal,breaks=breaks, axes=TRUE,lwd=0.1)
plot(boston.tr[, "predLex"],pal=pal,breaks=breaks, axes=TRUE,lwd=0.1)
```

```
plot(boston.tr[, "predCAR"],pal=pal,breaks=breaks, axes=TRUE,lwd=0.1)

predLm_samp <-resLm$samples$fitted
predLex_samp<-resLex$samples$fitted
predLm_95CI <-apply(predLm_samp ,2,function(x) quantile(x, probs=c(0.025, 0.975)))
predLex_95CI<-apply(predLex_samp,2,function(x) quantile(x, probs=c(0.025, 0.975)))

predLm_95CI[,1:8]
predLex_95CI[,1:8]
```

④ m_Section 16 (INLA)

2025/9/8: 最新版の INLA パッケージのバグで動作しない場合がありますが解消されたようです

```
##### Install inla
```

```
##### Inla のインストールについては下記も参照ください：
```

```
## https://www.r-inla.org/download-install
```

```
## --- 最小追加: タイムアウト延長 & 安定版優先→テスト版の順で自動トライ ---
```

```
options(timeout = 600)
```

```
options(download.file.method = "libcurl")
```

```
if (!requireNamespace("INLA", quietly = TRUE)) {
```

```
  if (!requireNamespace("BiocManager", quietly = TRUE))
```

```
    install.packages("BiocManager")
```

```
  BiocManager::install(c("graph", "Rgraphviz"), dep = TRUE, update = FALSE, ask = FALSE)
```

```
  repos_stable <- c(getOption("repos"), INLA = "https://inla.r-inla-download.org/R/stable")
```

```
  repos_testing <- c(getOption("repos"), INLA = "https://inla.r-inla-download.org/R/testing")
```

```
  ok <- TRUE
```

```
  tryCatch(install.packages("INLA", repos = repos_stable, dep = TRUE, type = "win.binary"),
```

```
    error = function(e) ok <- FALSE)
```

```
  if (!ok) {
```

```
    install.packages("INLA", repos = repos_testing, dep = TRUE, type = "win.binary")
```

```
  }
```

```
}
```

```
## （参考・元の記述：必要なら使う）
```

```
## install.packages("INLA", repos=c(getOption("repos"), INLA="https://inla.r-inla-download.org/R/testing"), dep=TRUE)
```

```
## or install.packages("INLA", repos=c(getOption("repos"), INLA="https://inla.r-inla-
```

```

download.org/R/stable"), dep=TRUE)

### For Windows
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
BiocManager::install(c("graph", "Rgraphviz"), dep=TRUE, update = FALSE, ask =
FALSE)

#####
library(INLA)
names(inla.models())$likelihood
names(inla.models())$latent
inla.list.models()

#####
##### example 1: disease mapping #####
#####

## 必要最小パッケージを自動導入（未インストール時のみ）
to_install <- setdiff(c("spdep", "CARBayesdata", "sf", "RColorBrewer", "fmesher"),
                      rownames(installed.packages()))
if (length(to_install)) install.packages(to_install, repos = "https://cloud.r-project.org",
type = "win.binary")

#install.packages( c( "spdep", "CARBayesdata", "sf", "RColorBrewer", fmesher ) )
library(spdep);library(CARBayesdata);library(RColorBrewer);library(fmesher);library(sf)

##### pollutionhealth data
data(pollutionhealthdata)

##### Adjacency matrix
data(GGHB.IZ)
W.nb <- poly2nb(GGHB.IZ)
W <- nb2mat(W.nb, style = "B")

##### Poisson regression

```

```

formula      <- observed ~ offset(log(expected)) + jsa + price + pm10
res          <- inla(formula=formula, data=pollutionhealthdata, family="poisson",
                    control.compute = list(dic = TRUE))

summary(res)

#### Poisson CAR model with group effects
pollutionhealthdata$ID <- as.integer(pollutionhealthdata$IZ) # zone ID
formula      <- observed ~ pm10 + jsa + offset(log(expected)) +
  f(ID, model = "besag", graph= W) + f(year, model="iid")
res          <- inla(formula=formula, data=pollutionhealthdata, family="nbinomial",
                    control.compute = list(dic = TRUE))

summary(res)

#### Estimated group effects
res$summary.random[[1]][1:4,]
res$summary.random[[2]]

#### Predicted values
pred         <- res$summary.fitted.values
pred[1:4,]

#### Plot the predicted value (posterior mean)
year         <- 2011
dat2011      <- pollutionhealthdata[pollutionhealthdata$year==year,]
dat2011$pred <- pred$mean[pollutionhealthdata$year==year]

data(GGHB.IZ)
dat          <- merge(x=GGHB.IZ, y=dat2011, by="IZ", all.x=FALSE, duplicateGeoms =
TRUE)
dat2         <- st_as_sf(dat)
nc           <- 11
breaks       <- seq(10, 200, len=nc+1)
pal          <- rev(brewer.pal(nc, "RdYlBu"))
plot(dat2[, c("observed", "pred")], pal=pal, breaks=breaks, axes=TRUE, lwd=0.1)

```

```
#####
##### example 2: spatial prediction #####
#####

# install.packages( c("gstat", "sf") )
library(sf); library(sp) #library(gstat);

##### Meuse data
data(meuse)
data(meuse.grid)

y      <- meuse$zinc   # 最小修正：ベクトルに明示（実行安定のため）
x      <- meuse[,c("dist", "ffreq")]
coords <- as.matrix( meuse[,c("x", "y")] )

x0      <- meuse.grid[,c("dist", "ffreq")]
coords0 <- as.matrix( meuse.grid[,c("x", "y")] )

##### Generate mesh
mesh1      <- inla.mesh.2d( loc=coords, max.edge = c(1000,1000))
mesh2      <- inla.mesh.2d( loc=coords, max.edge = c(300,1000))
mesh3      <- inla.mesh.2d( loc=coords, max.edge = c(300,300))
mesh4      <- inla.mesh.2d( loc=coords, max.edge = c(300,1000),cutoff=200)
mesh5      <- inla.mesh.2d( loc=coords, max.edge = c(300,1000),offset=c(-0.3,-
0.1))
mesh6      <- inla.mesh.2d( loc=coords, max.edge = c(300,1000),offset=c(-0.1,-
0.3))

plot(mesh1, asp=1);points(coords, col="red",pch=20,cex=0.7)
plot(mesh2, asp=1);points(coords, col="red",pch=20,cex=0.7)
plot(mesh3, asp=1);points(coords, col="red",pch=20,cex=0.7)
plot(mesh4, asp=1);points(coords, col="red",pch=20,cex=0.7)
plot(mesh5, asp=1);points(coords, col="red",pch=20,cex=0.7)
plot(mesh6, asp=1);points(coords, col="red",pch=20,cex=0.7)
```

```
#####
##### Conventional geostatistical modeling #####

##### Pre-processing
spde      <- inla.spde2.matern(mesh=mesh2, alpha=2) # 最小修正：INLA 仕様に合
           せ整数

A          <- inla.spde.make.A( mesh=mesh2, loc=coords )
stk        <- inla.stack(data=list(y=y), A=list(1, A), tag="obs",
                        effects=list(data.frame(intercept=1, dist=x$dist, ffreq=x$ffreq),
s=1:spde$n.spde))

A0         <- inla.spde.make.A(mesh=mesh2, loc=coords0)
stk0       <- inla.stack(data=list(y=NA), A=list(1, A0), tag="mis",
                        effects=list( data.frame(intercept=1, dist=x0$dist,
ffreq=x0$ffreq),s=1:spde$n.spde))

stk.full<- inla.stack(stk, stk0)
stk.data<- inla.stack.data(stk.full)
stk.A     <- inla.stack.A(stk.full)

##### Geostatistical modeling and prediction
mod      <- inla( log(y) ~ 0 + intercept + dist + ffreq + f(s, model=spde),
data=stk.data,
                control.predictor=list( A=stk.A ), family="gaussian")

mis.ind<- inla.stack.index(stk.full, tag="mis")$data
pred     <- mod$summary.fitted.values[mis.ind, "mean"]
pred_sd  <- mod$summary.fitted.values[mis.ind, "sd"]

##### Plot the predicted values
meuse.grid_sf      <- st_as_sf(meuse.grid, coords = c("x", "y"), crs = 28992)
meuse.grid_sf$pred  <- pred
meuse.grid_sf$pred_sd<-pred_sd
plot(meuse.grid_sf[, "pred"], axes=TRUE, pch=15)
plot(meuse.grid_sf[, "pred_sd"], axes=TRUE, pch=15)
```

```
#####
```

```
##### Geostatistical modeling considering physical barrier #####
```

```
##### [a] Constrained by the Meuse river
```

```
xran      <- range(mesh2$loc[,1])
yran      <- range(mesh2$loc[,2])
bcoords   <- rbind(c(xran[1], yran[1]),c(xran[2], yran[1]),
                  c(xran[2], yran[2]),c(xran[1], yran[2]),
                  c(xran[1], yran[1]))
polygon   <- st_sfc(st_polygon(list(bcoords)))
polygon_sf <- st_make_valid(st_sf(geometry = polygon))

data(meuse.riv)
meuse_sf  <- st_sfc(st_polygon(list(meuse.riv)))
meuse_sf  <- st_sf(geometry = meuse_sf)
meuse_sf2 <- st_difference(polygon_sf, meuse_sf)
mesh_b1   <- inla.mesh.2d( loc=coords, max.edge = 300, boundary=meuse_sf2)
```

```
##### [b] Constrained by the study area
```

```
meuse.grid_sp <-meuse.grid
coordinates(meuse.grid_sp) = c("x", "y")
gridded(meuse.grid_sp) <- TRUE
meuse.grid_sp <- as(meuse.grid_sp, "SpatialPolygons")
meuse.grid_sf <- st_as_sf(meuse.grid_sp)
meuse.area_sf <- st_union(meuse.grid_sf)
mesh_b2       <- inla.mesh.2d(loc = coords, max.edge = 300, boundary =
meuse.area_sf)
```

```
##### Spatial prediction under [a] or [b] #####
```

```
##### Pre-processing
```

```
mesh2      <-mesh_b1 ## or mesh_b2
plot(mesh2, asp=1);points(coords, col="red",pch=20,cex=0.7)
spde<- inla.spde2.matern(mesh=mesh2, alpha=2) # 最小修正：整数
```



```

A      <- inla.spde.make.A( mesh=mesh2, loc=coords)
stk    <- inla.stack(data=list(y=y), A=list(1, A), tag="obs",
                    effects=list(data.frame(intercept=1,dist=x$dist, ffreq=x$ffreq),
s=1:spde$n.spde))

A0      <- inla.spde.make.A(mesh=mesh2, loc=coords0)
stk0    <- inla.stack(data=list(y=NA), A=list(1, A0), tag="mis",
                    effects=list( data.frame(intercept=1,dist=x0$dist, ffreq=x0$ffreq),
s=1:spde$n.spde))

stk.full <- inla.stack(stk, stk0)
stk.data <- inla.stack.data(stk.full)
stk.A    <- inla.stack.A(stk.full)

##### Geostatistical modeling and prediction
mod      <- inla( log(y) ~ 0 + intercept + dist + ffreq + f(s, model=spde),
data=stk.data,
                    control.predictor=list( A=stk.A),family="gaussian")

mis.ind<- inla.stack.index(stk.full, tag="mis")$data
pred     <- mod$summary.fitted.values[mis.ind, "mean"]
pred_sd<- mod$summary.fitted.values[mis.ind, "sd"]
meuse.grid_sf$pred2    <-pred
meuse.grid_sf$pred2_sd<-pred_sd

##### Plot the predicted values
plot(meuse.grid_sf[, "pred2"], axes=TRUE, pch=15,border=NA)
plot(meuse.grid_sf[, "pred2_sd"], axes=TRUE, pch=15,border=NA)

```